==========

CAPSTONE PROJECT

NOTEBOOK 39: FULL-161 RARE-TAIL PREPARATION

==========

Purpose:

This notebook moves the project from the candidate-150 phase

toward the full all-genre objective by auditing and preparing

the remaining rare-tail labels.

The goal is to:

1. Load the full 161-label inventory and modelling tables

2. Separate candidate labels from rare-tail labels

3. Audit rare-tail label coverage across train/validation/test

4. Build rare-tail track subsets and track-label pair tables

5. Group rare-tail labels by parent/root genres

6. Create a full all-genre strategy table for the next phase

==========

In [1]:

```
# ============================================================
# 1. IMPORT LIBRARIES
# ============================================================

import os
import numpy as np
import pandas as pd

SEED = 42
np.random.seed(SEED)

print("Seed set to:", SEED)
```

```
Seed set to: 42
```

In [2]:

```
# ============================================================
# 2. LOAD CORE TABLES
# ============================================================

full_label_summary_df = pd.read_csv("../data/processed/full_label_summary.csv")
candidate_label_summary_df =
pd.read_csv("../data/processed/candidate_label_summary.csv")
genre_inventory_df = pd.read_csv("../data/processed/full_genre_inventory.csv")
full_master_df = pd.read_csv("../data/processed/multilabel_full_master_table.csv")

print("Full label summary shape:", full_label_summary_df.shape)
print("Candidate label summary shape:", candidate_label_summary_df.shape)
print("Genre inventory shape:", genre_inventory_df.shape)
print("Full master table shape:", full_master_df.shape)

display(full_label_summary_df.head())
```

```
display(candidate_label_summary_df.head())
display(genre_inventory_df.head())
display(full_master_df.head())
```

```
Full label summary shape: (163, 3)
Candidate label summary shape: (150, 3)
Genre inventory shape: (163, 11)
Full master table shape: (81574, 170)
```

	genre_id	genre_name	total_count
0	38	Experimental	35903
1	15	Electronic	28099
2	12	Rock	25820
3	1235	Instrumental	13588
4	10	Pop	12659

	genre_id	genre_name	total_count
0	38	Experimental	35903
1	15	Electronic	28099
2	12	Rock	25820
3	1235	Instrumental	13588
4	10	Pop	12659

	genre_id	genre_name	parent_id	parent_name	root_genre_id	root_genre_name	top_level_f
0	38	Experimental	NaN	NaN	38	Experimental	
1	15	Electronic	NaN	NaN	15	Electronic	
2	12	Rock	NaN	NaN	12	Rock	
3	1235	Instrumental	NaN	NaN	1235	Instrumental	1
4	10	Pop	NaN	NaN	10	Pop	

	track_id	split	subset	genre_top	title	audio_path	a
0	20	training	large	NaN	Spiritual Level	../data/raw/audio/fma_large\000\000020.mp3	
1	26	training	large	NaN	Where is your Love?	../data/raw/audio/fma_large\000\000026.mp3	
2	30	training	large	NaN	Too Happy	../data/raw/audio/fma_large\000\000030.mp3	
3	46	training	large	NaN	Yosemite	../data/raw/audio/fma_large\000\000046.mp3	
4	48	training	large	NaN	Light of Light	../data/raw/audio/fma_large\000\000048.mp3	

5 rows × 170 columns

In [3]:

```
# ============================================================
# 3. IDENTIFY FULL, CANDIDATE, AND RARE-TAIL LABEL SETS
# ============================================================

full_label_ids = set(full_label_summary_df["genre_id"].astype(int).tolist())
candidate_label_ids =
set(candidate_label_summary_df["genre_id"].astype(int).tolist())

rare_tail_label_ids = sorted(list(full_label_ids - candidate_label_ids))

print("Total full labels:", len(full_label_ids))
print("Total candidate labels:", len(candidate_label_ids))
print("Total rare-tail labels:", len(rare_tail_label_ids))
print("Rare-tail label IDs:", rare_tail_label_ids)
```

```
Total full labels: 163
Total candidate labels: 150
Total rare-tail labels: 13
Rare-tail label IDs: [173, 174, 175, 176, 178, 189, 374, 377, 465, 493, 808, 1032,
1060]
```

In [4]:

```
# ============================================================
# 4. BUILD LOOKUP MAPS
# ============================================================

inventory = genre_inventory_df.copy()

inventory["genre_id"] = inventory["genre_id"].astype(int)

# Ensure consistent typing for optional hierarchy columns
for col in ["parent_id", "root_genre_id", "top_level_flag"]:
```

```
    if col in inventory.columns:
        inventory[col] = pd.to_numeric(inventory[col], errors="coerce")

genre_name_map = dict(zip(inventory["genre_id"], inventory["genre_name"]))

parent_name_map = dict(zip(inventory["genre_id"], inventory.get("parent_name",
pd.Series([None] * len(inventory)))))
root_name_map = dict(zip(inventory["genre_id"], inventory.get("root_genre_name",
pd.Series([None] * len(inventory)))))
parent_id_map = dict(zip(inventory["genre_id"], inventory.get("parent_id",
pd.Series([np.nan] * len(inventory)))))
root_id_map = dict(zip(inventory["genre_id"], inventory.get("root_genre_id",
pd.Series([np.nan] * len(inventory)))))

print("Lookup maps created.")
```

```
Lookup maps created.
```

In [5]:

```
# ============================================================
# 5. IDENTIFY FULL LABEL COLUMNS IN MASTER TABLE
# ============================================================

full_label_cols = [
    col for col in full_master_df.columns
    if col.startswith("genre_") and col.replace("genre_", "").isdigit()
]

full_label_col_ids = sorted([int(col.replace("genre_", "")) for col in
full_label_cols])

rare_tail_label_cols = [f"genre_{gid}" for gid in rare_tail_label_ids if
f"genre_{gid}" in full_master_df.columns]
candidate_label_cols = [f"genre_{gid}" for gid in sorted(candidate_label_ids) if
f"genre_{gid}" in full_master_df.columns]

print("Number of full label columns found:", len(full_label_cols))
print("Number of candidate label columns found:", len(candidate_label_cols))
print("Number of rare-tail label columns found:", len(rare_tail_label_cols))
print("First few rare-tail label columns:", rare_tail_label_cols[:10])
```

```
Number of full label columns found: 163
Number of candidate label columns found: 150
Number of rare-tail label columns found: 13
First few rare-tail label columns: ['genre_173', 'genre_174', 'genre_175',
'genre_176', 'genre_178', 'genre_189', 'genre_374', 'genre_377', 'genre_465',
'genre_493']
```

In [6]:

```
# ============================================================
# 6. BASIC RARE-TAIL COVERAGE COUNTS
# ============================================================
```

```
split_names = ["training", "validation", "test"]

rare_tail_rows = []

for gid in rare_tail_label_ids:
    col = f"genre_{gid}"
    if col not in full_master_df.columns:
        continue

    row = {
        "genre_id": gid,
        "genre_name": genre_name_map.get(gid),
        "parent_id": parent_id_map.get(gid),
        "parent_name": parent_name_map.get(gid),
        "root_genre_id": root_id_map.get(gid),
        "root_genre_name": root_name_map.get(gid),
    }

    total_count = int(full_master_df[col].sum())
    row["total_count"] = total_count

    for split in split_names:
        split_count = int(full_master_df.loc[full_master_df["split"] == split,
col].sum())
        row[f"{split}_count"] = split_count

    rare_tail_rows.append(row)

rare_tail_summary_df = pd.DataFrame(rare_tail_rows).sort_values(
    ["total_count", "training_count", "validation_count", "test_count"],
    ascending=False
).reset_index(drop=True)

print("Rare-tail summary shape:", rare_tail_summary_df.shape)
display(rare_tail_summary_df)
```

```
Rare-tail summary shape: (13, 10)
```

	genre_id	genre_name	parent_id	parent_name	root_genre_id	root_genre_name	total_cou
0	176	Pacific	2.0	International	2	International	
1	1060	Tango	46.0	Latin America	2	International	
2	465	Musical Theater	20.0	Spoken	20	Spoken	
3	189	Talk Radio	65.0	Radio	20	Spoken	
4	1032	Turkish	102.0	Middle East	2	International	
5	174	South Indian Traditional	86.0	Indian	2	International	
6	374	Banter	20.0	Spoken	20	Spoken	
7	173	N. Indian Traditional	86.0	Indian	2	International	
8	493	Western Swing	651.0	Country & Western	9	Country	
9	377	Deep Funk	19.0	Funk	14	Soul-RnB	
10	808	Salsa	46.0	Latin America	2	International	
11	175	Bollywood	86.0	Indian	2	International	
12	178	Be-Bop	4.0	Jazz	4	Jazz	

In [7]:

```
# ============================================================
# 7. CREATE TRAINABILITY / FEASIBILITY FLAGS
# ============================================================

def classify_tail_row(row):
    train_count = row["training_count"]
    val_count = row["validation_count"]
    test_count = row["test_count"]

    if train_count == 0:
        return "No train support"
    if train_count > 0 and val_count == 0 and test_count == 0:
        return "Train only"
    if train_count > 0 and (val_count == 0 or test_count == 0):
        return "Partial split support"
    if train_count < 10:
        return "Extremely scarce"
    if train_count < 25:
        return "Very scarce"
    return "Tail-model feasible"

rare_tail_summary_df["feasibility_status"] =
rare_tail_summary_df.apply(classify_tail_row, axis=1)
```

```
print("Rare-tail feasibility summary:")
display(
    rare_tail_summary_df[
        [
            "genre_id", "genre_name", "training_count", "validation_count",
            "test_count", "total_count", "feasibility_status"
        ]
    ]
)
```

```
Rare-tail feasibility summary:
```

	genre_id	genre_name	training_count	validation_count	test_count	total_count	feasibility
0	176	Pacific	17	2	4	23	Very
1	1060	Tango	5	6	12	23	Extremely
2	465	Musical Theater	4	4	10	18	Extremely
3	189	Talk Radio	13	1	1	15	Very
4	1032	Turkish	10	0	5	15	Part
5	174	South Indian Traditional	0	1	14	15	No train
6	374	Banter	5	0	0	5	Tra
7	173	N. Indian Traditional	3	0	1	4	Part
8	493	Western Swing	1	1	2	4	Extremely
9	377	Deep Funk	1	0	0	1	Tra
10	808	Salsa	1	0	0	1	Tra
11	175	Bollywood	0	0	0	0	No train
12	178	Be-Bop	0	0	0	0	No train

In [8]:

```
# ============================================================
# 8. BUILD RARE-TAIL TRACK SUBSET
# ============================================================

full_master_df["rare_tail_label_count"] =
full_master_df[rare_tail_label_cols].sum(axis=1)

meta_cols = [
    col for col in [
        "track_id", "split", "subset", "genre_top", "title",
```

```
        "audio_path", "audio_exists"
    ] if col in full_master_df.columns
]

rare_tail_tracks_df = full_master_df.loc[
    full_master_df["rare_tail_label_count"] > 0,
    meta_cols + rare_tail_label_cols
].copy().reset_index(drop=True)

rare_tail_tracks_df["rare_tail_label_count"] =
rare_tail_tracks_df[rare_tail_label_cols].sum(axis=1)

print("Rare-tail track subset shape:", rare_tail_tracks_df.shape)

print("Rare-tail track split distribution:")
print(rare_tail_tracks_df["split"].value_counts())

display(rare_tail_tracks_df.head())
```

```
Rare-tail track subset shape: (124, 21)
Rare-tail track split distribution:
split
training      60
test          49
validation    15
Name: count, dtype: int64
```

	track_id	split	subset	genre_top	title	audio_path
0	13077	validation	large	NaN	Thanam-Kalyani	../data/raw/audio/fma_large\013\013077.mp3
1	16930	test	large	NaN	Open Session	../data/raw/audio/fma_large\016\016930.mp3
2	19777	validation	large	NaN	Custer: "If I Were An Indian..."	../data/raw/audio/fma_large\019\019777.mp3
3	19778	validation	large	NaN	Custer's Ghost To Sitting Bull	../data/raw/audio/fma_large\019\019778.mp3
4	19779	validation	large	NaN	Sitting Bull: "Do You Know Who I Am?"	../data/raw/audio/fma_large\019\019779.mp3

5 rows × 21 columns

In [9]:

```
# ============================================================
# 9. BUILD EXPLODED RARE-TAIL TRACK-LABEL PAIRS
# ============================================================

pair_rows = []

for _, row in rare_tail_tracks_df.iterrows():
    track_id = int(row["track_id"])

    for col in rare_tail_label_cols:
        if int(row[col]) == 1:
            gid = int(col.replace("genre_", ""))

            pair_rows.append({
                "track_id": track_id,
                "split": row["split"],
                "subset": row.get("subset", None),
                "genre_top": row.get("genre_top", None),
                "title": row.get("title", None),
                "audio_path": row.get("audio_path", None),
                "audio_exists": row.get("audio_exists", None),
                "rare_tail_genre_id": gid,
                "rare_tail_genre_name": genre_name_map.get(gid),
                "parent_id": parent_id_map.get(gid),
                "parent_name": parent_name_map.get(gid),
                "root_genre_id": root_id_map.get(gid),
                "root_genre_name": root_name_map.get(gid)
            })

rare_tail_pairs_df = pd.DataFrame(pair_rows)

print("Rare-tail track-label pairs shape:", rare_tail_pairs_df.shape)
display(rare_tail_pairs_df.head(20))
```

```
Rare-tail track-label pairs shape: (124, 13)
```

	track_id	split	subset	genre_top	title	audio_p
0	13077	validation	large	NaN	Thanam-Kalyani	../data/raw/audio/fma_large\013\013077.r
1	16930	test	large	NaN	Open Session	../data/raw/audio/fma_large\016\016930.r
2	19777	validation	large	NaN	Custer: "If I Were An Indian..."	../data/raw/audio/fma_large\019\019777.r
3	19778	validation	large	NaN	Custer's Ghost To Sitting Bull	../data/raw/audio/fma_large\019\019778.r
4	19779	validation	large	NaN	Sitting Bull: "Do You Know Who I Am?"	../data/raw/audio/fma_large\019\019779.r
5	19780	validation	large	NaN	Sun Dance / Battle Of The Greasy Grass River	../data/raw/audio/fma_large\019\019780.r
6	21268	training	large	NaN	Seed	../data/raw/audio/fma_large\021\021268.r
7	21269	training	large	NaN	Bedtime	../data/raw/audio/fma_large\021\021269.r
8	21270	training	large	NaN	A Leaf's Lament	../data/raw/audio/fma_large\021\021270.r
9	21271	training	large	NaN	A Bird Named Bob	../data/raw/audio/fma_large\021\021271.r
10	21272	training	large	NaN	Crow Berry	../data/raw/audio/fma_large\021\021272.r
11	21273	training	large	NaN	Moon II	../data/raw/audio/fma_large\021\021273.r
12	21274	training	large	NaN	The Whirlwinds Grandmother	../data/raw/audio/fma_large\021\021274.r
13	23153	training	large	NaN	Embellir (with Les Gauchers Quintet)	../data/raw/audio/fma_large\023\023153.r
14	27962	training	large	NaN	(c) Warrior Woman That I Am	../data/raw/audio/fma_large\027\027962.r
15	30180	test	large	NaN	Conic Sections	../data/raw/audio/fma_large\030\030180.r
16	30181	test	large	NaN	Elliptically	../data/raw/audio/fma_large\030\030181.r
17	30182	test	large	NaN	Hyperboli	../data/raw/audio/fma_large\030\030182.r

	track_id	split	subset	genre_top	title	audio_p
18	30183	test	large	NaN	Directrix	../data/raw/audio/fma_large\030\030183.r
19	30184	test	large	NaN	So It Goes	../data/raw/audio/fma_large\030\030184.r

In [10]:

```
# ============================================================
# 10. RARE-TAIL ROOT-GENRE SUMMARY
# ============================================================

rare_tail_root_summary_df = (
    rare_tail_pairs_df.groupby(["root_genre_id", "root_genre_name"])
    .agg(
        rare_tail_label_instances=("rare_tail_genre_id", "count"),
        unique_rare_tail_labels=("rare_tail_genre_id", "nunique"),
        unique_tracks=("track_id", "nunique")
    )
    .reset_index()
    .sort_values(
        ["rare_tail_label_instances", "unique_rare_tail_labels", "unique_tracks"],
        ascending=False
    )
    .reset_index(drop=True)
)

print("Rare-tail root summary:")
display(rare_tail_root_summary_df)
```

```
Rare-tail root summary:
```

	root_genre_id	root_genre_name	rare_tail_label_instances	unique_rare_tail_labels	unique_trac
0	2	International	81	6	
1	20	Spoken	38	3	
2	9	Country	4	1	
3	14	Soul-RnB	1	1	

In [11]:

```
# ============================================================
# 11. BUILD FULL ALL-GENRE STRATEGY TABLE
# ============================================================

candidate_set = set(candidate_label_ids)

# Merge counts into inventory
strategy_df = inventory.copy()
```

```
full_count_map = dict(zip(full_label_summary_df["genre_id"].astype(int),
full_label_summary_df["total_count"]))
strategy_df["total_count"] =
strategy_df["genre_id"].map(full_count_map).fillna(0).astype(int)

split_count_lookup = rare_tail_summary_df.set_index("genre_id")[
    ["training_count", "validation_count", "test_count", "feasibility_status"]
].to_dict(orient="index")

training_counts = []
validation_counts = []
test_counts = []
feasibility_list = []
stage_roles = []
recommended_actions = []

for gid in strategy_df["genre_id"].astype(int):
    if gid in candidate_set:
        training_counts.append(np.nan)
        validation_counts.append(np.nan)
        test_counts.append(np.nan)
        feasibility_list.append("Direct candidate label")
        stage_roles.append("Stage 1")
        recommended_actions.append("Direct modelling in candidate-150 system")
    else:
        info = split_count_lookup.get(gid, None)
        if info is None:
            training_counts.append(0)
            validation_counts.append(0)
            test_counts.append(0)
            feasibility_list.append("Unused / unavailable")
            stage_roles.append("Stage 2")
            recommended_actions.append("Inventory only")
        else:
            training_counts.append(info["training_count"])
            validation_counts.append(info["validation_count"])
            test_counts.append(info["test_count"])
            feasibility_list.append(info["feasibility_status"])
            stage_roles.append("Stage 2")

            status = info["feasibility_status"]
            if status == "Tail-model feasible":
                recommended_actions.append("Candidate for specialized rare-tail
model")
            elif status in ["Very scarce", "Extremely scarce", "Partial split
support"]:
                recommended_actions.append("Use hierarchy/fallback/few-shot
handling")
            elif status == "Train only":
                recommended_actions.append("Weak support; use hierarchy or
retrieval-style fallback")
            elif status == "No train support":
                recommended_actions.append("Cannot train directly; use
hierarchy/inventory fallback")
            else:
                recommended_actions.append("Review manually")
```

```
strategy_df["training_count"] = training_counts
strategy_df["validation_count"] = validation_counts
strategy_df["test_count"] = test_counts
strategy_df["feasibility_status"] = feasibility_list
strategy_df["stage_role"] = stage_roles
strategy_df["recommended_action"] = recommended_actions

strategy_df = strategy_df.sort_values(
    ["stage_role", "total_count", "genre_id"],
    ascending=[True, False, True]
).reset_index(drop=True)

print("Full all-genre strategy table:")
display(
    strategy_df[
        [
            "genre_id", "genre_name", "parent_name", "root_genre_name",
            "total_count", "stage_role", "feasibility_status", "recommended_action"
        ]
    ].head(50)
)
```

```
Full all-genre strategy table:
```

	genre_id	genre_name	parent_name	root_genre_name	total_count	stage_role	feasibilit
0	38	Experimental	NaN	Experimental	35903	Stage 1	Direct ca
1	15	Electronic	NaN	Electronic	28099	Stage 1	Direct ca
2	12	Rock	NaN	Rock	25820	Stage 1	Direct ca
3	1235	Instrumental	NaN	Instrumental	13588	Stage 1	Direct ca
4	10	Pop	NaN	Pop	12659	Stage 1	Direct ca
5	17	Folk	NaN	Folk	11187	Stage 1	Direct ca
6	1	Avant-Garde	Experimental	Experimental	8134	Stage 1	Direct ca
7	107	Ambient	Instrumental	Instrumental	6799	Stage 1	Direct ca
8	32	Noise	Experimental	Experimental	6781	Stage 1	Direct ca
9	76	Experimental Pop	Pop	Pop	6632	Stage 1	Direct ca
10	21	Hip-Hop	NaN	Hip-Hop	6188	Stage 1	Direct ca
11	25	Punk	Rock	Rock	5942	Stage 1	Direct ca
12	41	Electroacoustic	Experimental	Experimental	5793	Stage 1	Direct ca
13	27	Lo-Fi	Rock	Rock	5492	Stage 1	Direct ca
14	18	Soundtrack	Instrumental	Instrumental	5092	Stage 1	Direct ca
15	42	Ambient Electronic	Electronic	Electronic	4719	Stage 1	Direct ca
16	2	International	NaN	International	4253	Stage 1	Direct ca
17	66	Indie-Rock	Rock	Rock	4139	Stage 1	Direct ca
18	250	Improv	Experimental	Experimental	4031	Stage 1	Direct ca

	genre_id	genre_name	parent_name	root_genre_name	total_count	stage_role	feasibilit
19	4	Jazz	NaN	Jazz	3742	Stage 1	Direct ca
20	103	Singer-Songwriter	Folk	Folk	3588	Stage 1	Direct ca
21	5	Classical	NaN	Classical	3487	Stage 1	Direct ca
22	30	Field Recordings	Experimental	Experimental	3037	Stage 1	Direct ca
23	247	Musique Concrete	Experimental	Experimental	2830	Stage 1	Direct ca
24	236	IDM	Electronic	Electronic	2484	Stage 1	Direct ca
25	47	Drone	Experimental	Experimental	2340	Stage 1	Direct ca
26	183	Glitch	Electronic	Electronic	2260	Stage 1	Direct ca
27	85	Garage	Rock	Rock	2131	Stage 1	Direct ca
28	33	Psych-Folk	Folk	Folk	2123	Stage 1	Direct ca
29	70	Industrial	Rock	Rock	2058	Stage 1	Direct ca
30	45	Loud-Rock	Rock	Rock	1934	Stage 1	Direct ca
31	58	Psych-Rock	Rock	Rock	1878	Stage 1	Direct ca
32	9	Country	NaN	Country	1809	Stage 1	Direct ca
33	20	Spoken	NaN	Spoken	1758	Stage 1	Direct ca
34	3	Blues	NaN	Blues	1678	Stage 1	Direct ca
35	53	Noise-Rock	Loud-Rock	Rock	1677	Stage 1	Direct ca
36	224	Sound Collage	Experimental	Experimental	1673	Stage 1	Direct ca
37	362	Synth Pop	Pop	Pop	1623	Stage 1	Direct ca

	genre_id	genre_name	parent_name	root_genre_name	total_count	stage_role	feasibilit
38	74	Free-Jazz	Jazz	Jazz	1471	Stage 1	Direct ca
39	26	Post-Rock	Rock	Rock	1435	Stage 1	Direct ca
40	14	Soul-RnB	NaN	Soul-RnB	1345	Stage 1	Direct ca
41	181	Techno	Electronic	Electronic	1338	Stage 1	Direct ca
42	125	Unclassifiable	Experimental	Experimental	1337	Stage 1	Direct ca
43	514	Sound Art	Experimental	Experimental	1294	Stage 1	Direct ca
44	495	Downtempo	Electronic	Electronic	1270	Stage 1	Direct ca
45	456	Minimalism	Experimental	Experimental	1258	Stage 1	Direct ca
46	94	Freak-Folk	Folk	Folk	1217	Stage 1	Direct ca
47	89	Post-Punk	Punk	Rock	1213	Stage 1	Direct ca
48	297	Chip Music	Electronic	Electronic	1179	Stage 1	Direct ca
49	659	Contemporary Classical	Classical	Classical	1155	Stage 1	Direct ca

In [12]:

```
# ============================================================
# 12. BUILD FINAL FULL-GENRE STATUS TABLE
# ============================================================

final_status_rows = []

final_status_rows.append({
    "Component": "Stage 1 candidate system",
    "Status": "Complete",
    "Details": "Candidate-150 structured, audio, and hybrid models have been
trained, validated, tested, and consolidated."
})

final_status_rows.append({
    "Component": "Full 161 inventory",
    "Status": "Complete",
    "Details": f"All used labels have been inventoried. Full label count =
```

```
{len(full_label_ids)}."
})

final_status_rows.append({
    "Component": "Rare-tail audit",
    "Status": "Complete",
    "Details": f"Rare-tail label count = {len(rare_tail_label_ids)}. Coverage by
split has been summarized."
})

tail_feasible_count = int((rare_tail_summary_df["feasibility_status"] == "Tail-
model feasible").sum())
tail_scarce_count = int(rare_tail_summary_df["feasibility_status"].isin(["Very
scarce", "Extremely scarce"]).sum())
tail_no_train_count = int((rare_tail_summary_df["feasibility_status"] == "No train
support").sum())

final_status_rows.append({
    "Component": "Rare-tail direct modelling readiness",
    "Status": "Partial",
    "Details": (
        f"{tail_feasible_count} rare-tail labels look directly model-feasible, "
        f"{tail_scarce_count} are scarce/very scarce, and "
        f"{tail_no_train_count} have no train support."
    )
})

final_status_rows.append({
    "Component": "Full all-genre pipeline",
    "Status": "Not yet complete",
    "Details": "The candidate-150 pipeline is complete, but stage-2 rare-tail
handling still needs to be built."
})

final_status_df = pd.DataFrame(final_status_rows)

print("Final full-genre status table:")
display(final_status_df)
```

```
Final full-genre status table:
```

	Component	Status	Details
0	Stage 1 candidate system	Complete	Candidate-150 structured, audio, and hybrid mo...
1	Full 161 inventory	Complete	All used labels have been inventoried. Full la...
2	Rare-tail audit	Complete	Rare-tail label count = 13. Coverage by split ...
3	Rare-tail direct modelling readiness	Partial	0 rare-tail labels look directly model-feasibl...
4	Full all-genre pipeline	Not yet complete	The candidate-150 pipeline is complete, but st...

In [13]:

```
# ============================================================
# 13. SAVE OUTPUTS
# ============================================================

os.makedirs("../data/processed", exist_ok=True)

rare_tail_summary_df.to_csv(
    "../data/processed/full161_rare_tail_summary.csv",
    index=False
)

rare_tail_tracks_df.to_csv(
    "../data/processed/full161_rare_tail_tracks.csv",
    index=False
)

rare_tail_pairs_df.to_csv(
    "../data/processed/full161_rare_tail_track_label_pairs.csv",
    index=False
)

rare_tail_root_summary_df.to_csv(
    "../data/processed/full161_rare_tail_root_summary.csv",
    index=False
)

strategy_df.to_csv(
    "../data/processed/full161_all_genre_strategy_table.csv",
    index=False
)

final_status_df.to_csv(
    "../data/processed/full161_project_status_table.csv",
    index=False
)

print("Saved full-161 rare-tail preparation outputs.")
```

```
```

```
Saved full-161 rare-tail preparation outputs.
```

In [14]:

```
# ============================================================
# 14. INTERPRETATION NOTES
# ============================================================

print("1. The candidate-150 system is finished, but the full all-genre system is not yet finished.")
print("2. The remaining work is the rare-tail stage that extends coverage from candidate labels to the full label inventory.")
print("3. This notebook prepares the remaining rare-tail labels for the next modelling stage.")
print("4. The next notebook should build a rare-tail modelling or fallback pipeline.")
print("5. After that, the project can move toward a true full all-genre inference pipeline.")
```

```
1. The candidate-150 system is finished, but the full all-genre system is not yet finished.
2. The remaining work is the rare-tail stage that extends coverage from candidate labels to the full label inventory.
3. This notebook prepares the remaining rare-tail labels for the next modelling stage.
4. The next notebook should build a rare-tail modelling or fallback pipeline.
5. After that, the project can move toward a true full all-genre inference pipeline.
```